Atividade 4

Random Neurons - um *ensemble* bagging-subespaço com Perceptrons de Bolso

Matheus Venancio Moreira Sales

24 de abril de 2026

1 Introdução

Random Forest é um dos *ensembles* mais bem sucedidos na literatura de aprendizado de máquina. Sua eficácia decorre de três ingredientes combinados: (i) *bagging*, isto é, a agregação de classificadores treinados em réplicas *bootstrap* dos dados; (ii) *random subspace*, em que cada componente enxerga apenas um subconjunto aleatório de atributos; e (iii) a escolha de árvores de decisão como classificador base, que são de baixo viés e alta variância — exatamente o perfil que o *bagging* sabe estabilizar. Uma pergunta natural é: o que acontece quando substituímos as árvores por modelos de viés oposto, classificadores *lineares*?

O presente trabalho implementa essa variante, a que damos o nome de *Random Neurons*, em que o classificador base é um **Perceptron de Bolso** estendido para problemas multiclasse via estratégia *One-vs-Rest* (OVR). Seguimos estritamente o roteiro proposto no enunciado:

(a) Para um dado $L \in \mathbb{N}$, obter L réplicas D_ℓ do conjunto de treino D por *bootstrap*;

(b) Para cada D_ℓ, sortear $k \in \{1, \ldots, n\}$ uniformemente e, em seguida, sortear sem reposição k índices dentre os atributos, armazenando tais índices em estrutura de acesso rápido;

(c) Treinar, sobre D_ℓ restrito aos k atributos sorteados, um classificador Perceptron de Bolso (estendido por OVR) com no máximo M iterações;

(d) Classificar um padrão $\mathbf{x} \in \mathbb{R}^n$ tomando a classe mais frequente entre as predições dos L componentes (votação majoritária).

O restante deste documento está organizado como segue. A Seção 2 revisa os blocos constituintes (Perceptron de Bolso e OVR). A Seção 3 descreve formalmente o *Random Neurons*. A Seção 4 apresenta a metodologia experimental adotada. A Seção 5 exibe os resultados numéricos e gráficos, e a Seção 6 traz a discussão crítica.

2 Fundamentação

2.1 Perceptron de Bolso

O Perceptron clássico de Rosenblatt busca um hiperplano separador por meio da atualização

$$\mathbf{w} \leftarrow \mathbf{w} + y_i \, \tilde{\mathbf{x}}_i, \tag{1}$$

aplicada a cada padrão $\tilde{\mathbf{x}}_i = [\mathbf{x}_i^\top, 1]^\top$ mal classificado, sendo $y_i \in \{-1, +1\}$. Se os dados forem linearmente separáveis, garante-se convergência em número finito de passos. No entanto, quando não há separabilidade linear, os pesos *oscilam indefinidamente*, nunca se estabilizando em uma boa solução.

O *Pocket Perceptron* remedia isso mantendo, a cada iteração, uma cópia "no bolso" do melhor vetor $\mathbf{w}^\star$ já observado, medido pelo número de classificações corretas sobre o conjunto de treino. Ao fim das M iterações, retorna-se $\mathbf{w}^\star$ em vez do último $\mathbf{w}$. O Algoritmo 1 resume o procedimento.

Algoritmo 1: Perceptron de Bolso

Input: Conjunto de treino $\{(\tilde{\mathbf{x}}_i, y_i)\}_{i=1}^m$, $y_i \in \{-1, +1\}$; número máximo de iterações M.
Output: Vetor de pesos $\mathbf{w}^\star$.

Inicializar $\mathbf{w}$ aleatoriamente; $\mathbf{w}^\star \leftarrow \mathbf{w}$; $E^\star \leftarrow \#\{i : \operatorname{sgn}(\mathbf{w}^\top \tilde{\mathbf{x}}_i) \neq y_i\}$
for $t \leftarrow 1$ **to** M **do**
  Selecionar aleatoriamente i tal que $\operatorname{sgn}(\mathbf{w}^\top \tilde{\mathbf{x}}_i) \neq y_i$
  if *não existe tal* i **then return** $\mathbf{w}$
  $\mathbf{w} \leftarrow \mathbf{w} + y_i\, \tilde{\mathbf{x}}_i$
  $E \leftarrow \#\{i : \operatorname{sgn}(\mathbf{w}^\top \tilde{\mathbf{x}}_i) \neq y_i\}$
  if $E < E^\star$ **then** $\mathbf{w}^\star \leftarrow \mathbf{w}$; $E^\star \leftarrow E$
end
return $\mathbf{w}^\star$

2.2 One-vs-Rest (OVR)

O Perceptron é inerentemente binário. Para um problema com c classes, a estratégia OVR treina c classificadores $\mathbf{w}_1, \ldots, \mathbf{w}_c$, o k-ésimo tomando a classe k como positiva e todas as demais como negativas. Na predição, atribuímos ao padrão $\mathbf{x}$ a classe associada à maior função de decisão:

$$\hat{y}(\mathbf{x}) \;=\; \arg\max_{k \in \{1,\ldots,c\}} \; \mathbf{w}_k^\top \tilde{\mathbf{x}}. \tag{2}$$

O uso da *decision function* em vez do sinal resolve naturalmente eventuais empates (casos em que dois ou mais classificadores "reivindicam" o padrão), escolhendo aquele com maior margem.

3 O algoritmo Random Neurons

Seja $D = \{(\mathbf{x}_i, y_i) \in \mathbb{R}^n \times \{1, \ldots, c\}\}_{i=1}^m$. O Algoritmo 2 descreve o treinamento e a predição do *Random Neurons*.

Algoritmo 2: Random Neurons

Input: D; L (número de componentes); M (iterações do Pocket).

Função *Fit*(D, L, M)**:**
  for $\ell \leftarrow 1$ **to** L **do**
    $D_\ell \leftarrow$ amostra *bootstrap* de D com reposição, $|D_\ell| = m$
    $k_\ell \leftarrow$ inteiro sorteado uniformemente em $\{1, 2, \ldots, n\}$
    $\mathcal{F}_\ell \leftarrow k_\ell$ índices sorteados sem reposição em $\{1, \ldots, n\}$
    $\mathbf{h}_\ell \leftarrow$ OVR-Pocket$(D_\ell|_{\mathcal{F}_\ell}, M)$
  end
  return $\{(\mathbf{h}_\ell, \mathcal{F}_\ell)\}_{\ell=1}^L$
fim
Função *Predict*$(\mathbf{x})$**:**
  for $\ell \leftarrow 1$ **to** L **do**
    $v_\ell \leftarrow \mathbf{h}_\ell(\mathbf{x}|_{\mathcal{F}_\ell})$
  end
  return $\arg\max_c \sum_{\ell=1}^L \mathbb{1}[v_\ell = c]$ `// voto majoritário`
fim

A chamada OVR-Pocket(D, M) treina um classificador OVR como descrito na Seção 2, em que cada componente binário é um Perceptron de Bolso com no máximo M iterações. A restrição $D_\ell|_{\mathcal{F}_\ell}$ denota o conjunto D_ℓ projetado apenas nas colunas indexadas por $\mathcal{F}_\ell$. Os índices sorteados são armazenados em uma lista paralela aos modelos, o que permite consulta $\mathcal{O}(1)$ na predição.

4 Metodologia experimental

Datasets. Foram utilizados os conjuntos públicos *Iris* ($m = 150$, $n = 4$, $c = 3$) e *Wine* ($m = 178$, $n = 13$, $c = 3$), ambos disponíveis no scikit-learn.

Divisão treino/teste. Para cada execução foi realizada uma divisão *aleatória estratificada* na proporção 70% / 30%, de modo a preservar a proporção original de classes em ambos os subconjuntos.

Pré-processamento. O *Random Neurons* foi alimentado com atributos padronizados ($\mu = 0$, $\sigma = 1$), com o `StandardScaler` ajustado *exclusivamente* no conjunto de treino. A padronização é importante porque o Perceptron é sensível à escala dos atributos: sem ela, os passos de atualização (1) seriam desbalanceados entre as dimensões. O *Random Forest*, por ser baseado em partições axiais, é invariante a transformações monotônicas e, portanto, não requer padronização.

Grid de parâmetros. Para o *Random Neurons*, variamos $L \in \{5, 10, 25, 50, 100\}$ e $M \in \{10, 50, 200\}$, totalizando 15 combinações. O *Random Forest* foi executado com L árvores (`n_estimators` $= L$), critério de Gini, `max_features` $=$' `sqrt`' e `bootstrap` = `True` — valores padrão razoáveis e amplamente adotados na literatura.

Robustez estatística. Cada configuração foi executada com 8 sementes diferentes (`random_state` $\in \{0, 1, \ldots, 7\}$), que controlam tanto a divisão treino/teste quanto a inicialização dos modelos. Reportamos média e desvio-padrão do percentual de acerto sobre o conjunto de teste.

5 Resultados

5.1 Tabelas de acurácia

As Tabelas 1 e 2 apresentam a acurácia média $\pm$ desvio-padrão (em %) sobre o conjunto de teste, para as 15 combinações de (L, M), nos conjuntos *Iris* e *Wine*, respectivamente. A coluna *Random Forest* reporta o baseline com o mesmo L (e é, portanto, constante ao longo de M).

Tabela 1: Acurácia de teste (%) no *Iris*. Média ± desvio-padrão em 8 sementes.

L	M	**Random Neurons**	**Random Forest**
5	10	$85{,}00 \pm 4{,}41$	$94{,}44 \pm 2{,}72$
5	50	$88{,}06 \pm 6{,}57$	$94{,}44 \pm 2{,}72$
5	200	$91{,}39 \pm 4{,}90$	$94{,}44 \pm 2{,}72$
10	10	$86{,}67 \pm 3{,}69$	$95{,}28 \pm 3{,}03$
10	50	$90{,}00 \pm 4{,}71$	$95{,}28 \pm 3{,}03$
10	200	$92{,}78 \pm 3{,}47$	$95{,}28 \pm 3{,}03$
25	10	$85{,}00 \pm 4{,}68$	$95{,}56 \pm 2{,}72$
25	50	$91{,}39 \pm 4{,}50$	$95{,}56 \pm 2{,}72$
25	200	$\mathbf{93{,}89 \pm 3{,}47}$	$95{,}56 \pm 2{,}72$
50	10	$86{,}39 \pm 3{,}23$	$95{,}28 \pm 2{,}82$
50	50	$92{,}22 \pm 4{,}01$	$95{,}28 \pm 2{,}82$
50	200	$93{,}89 \pm 3{,}09$	$95{,}28 \pm 2{,}82$
100	10	$86{,}67 \pm 3{,}85$	$95{,}83 \pm 2{,}59$
100	50	$91{,}67 \pm 4{,}41$	$95{,}83 \pm 2{,}59$
100	200	$93{,}61 \pm 3{,}59$	$95{,}83 \pm 2{,}59$

Tabela 2: Acurácia de teste (%) no *Wine*. Média ± desvio-padrão em 8 sementes.

L	M	**Random Neurons**	**Random Forest**
5	10	$95{,}37 \pm 4{,}14$	$95{,}14 \pm 2{,}92$
5	50	$96{,}53 \pm 2{,}99$	$95{,}14 \pm 2{,}92$
5	200	$96{,}99 \pm 2{,}77$	$95{,}14 \pm 2{,}92$
10	10	$97{,}45 \pm 1{,}59$	$97{,}45 \pm 2{,}06$
10	50	$98{,}15 \pm 1{,}60$	$97{,}45 \pm 2{,}06$
10	200	$\mathbf{99{,}07 \pm 1{,}31}$	$97{,}45 \pm 2{,}06$
25	10	$98{,}38 \pm 1{,}11$	$98{,}61 \pm 1{,}22$
25	50	$98{,}61 \pm 0{,}80$	$98{,}61 \pm 1{,}22$
25	200	$98{,}61 \pm 1{,}22$	$98{,}61 \pm 1{,}22$
50	10	$98{,}15 \pm 1{,}31$	$98{,}84 \pm 1{,}29$
50	50	$98{,}38 \pm 1{,}45$	$98{,}84 \pm 1{,}29$
50	200	$98{,}38 \pm 1{,}45$	$98{,}84 \pm 1{,}29$
100	10	$98{,}84 \pm 1{,}59$	$98{,}61 \pm 1{,}54$
100	50	$98{,}61 \pm 1{,}79$	$98{,}61 \pm 1{,}54$
100	200	$98{,}61 \pm 1{,}79$	$98{,}61 \pm 1{,}54$

5.2 Visualização dos resultados

A Figura 1 apresenta a acurácia em função de L para cada valor de M, juntamente com a curva do *Random Forest* (linha tracejada preta). As barras de erro correspondem ao desvio-padrão entre as 8 sementes. O eixo L está em escala logarítmica, para melhor visualizar o comportamento assintótico.

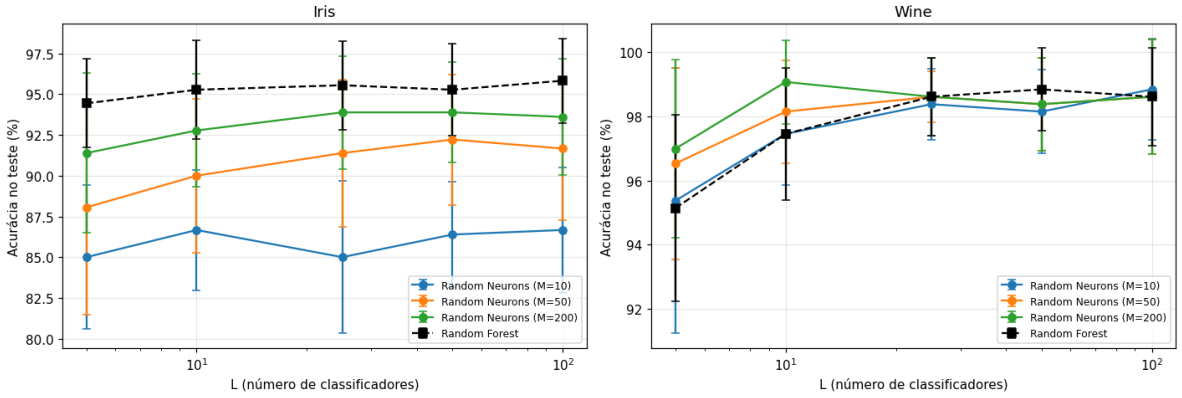

Figura 1: Acurácia de teste em função de L, por valor de M, para os conjuntos *Iris* (esquerda) e *Wine* (direita). As barras de erro representam o desvio-padrão em 8 sementes.

A Figura 2 mostra os mesmos resultados do *Random Neurons* em formato de mapa de calor, facilitando a identificação das configurações de melhor desempenho. A Figura 3 complementa essa visualização exibindo a *diferença* entre a acurácia do *Random Neurons* e a do *Random Forest* para cada (L, M), em pontos percentuais.

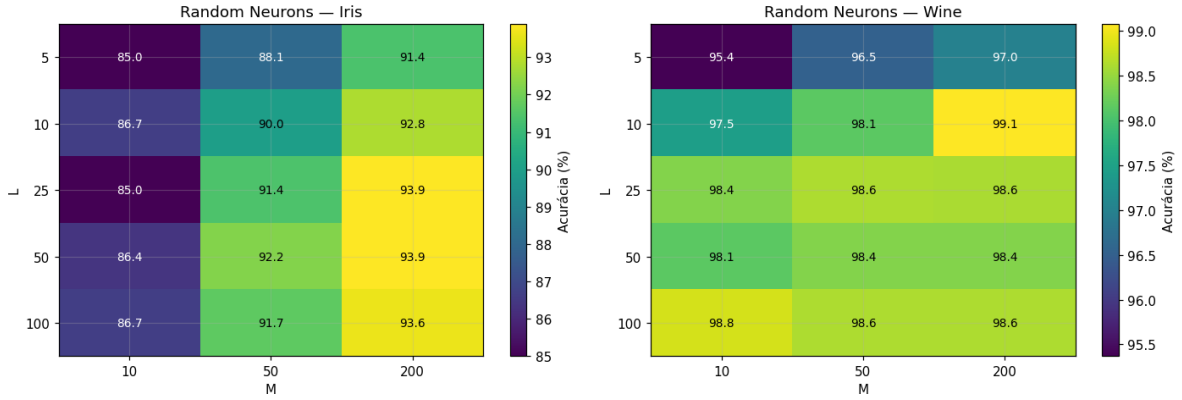

Figura 2: Mapa de calor da acurácia do *Random Neurons* (em %) na grade (L, M), para *Iris* (esquerda) e *Wine* (direita).

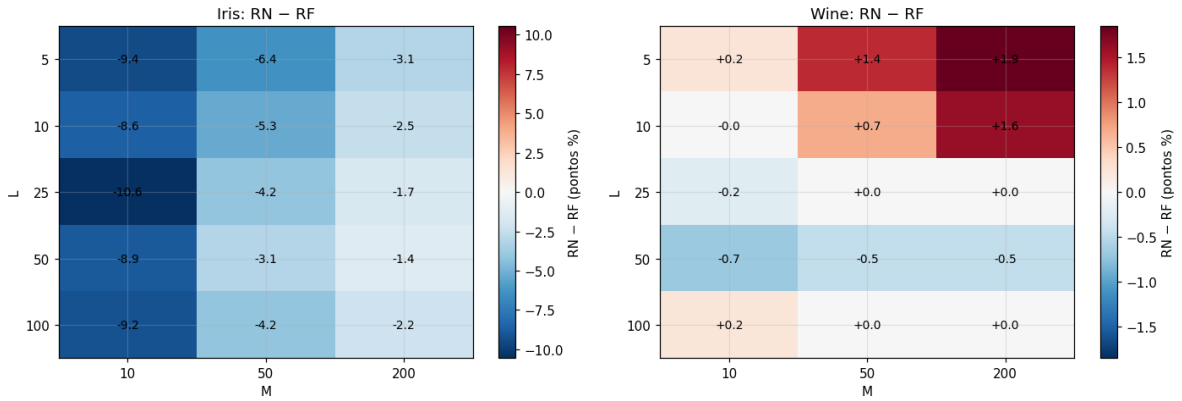

Figura 3: Diferença (*Random Neurons* − *Random Forest*), em pontos percentuais, para cada configuração (L, M). Tons avermelhados favorecem o *Random Neurons*; tons azulados, o *Random Forest*.

Finalmente, a Figura 4 compara a melhor configuração do *Random Neurons* encontrada em cada dataset diretamente com o *Random Forest* correspondente.

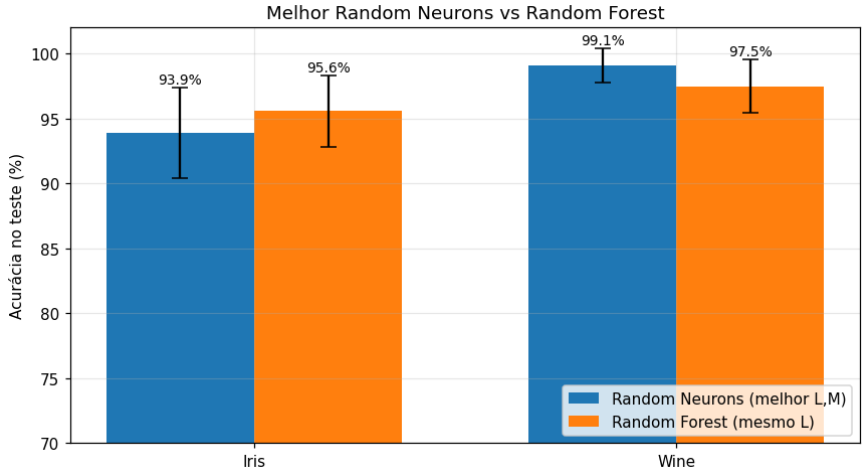

Figura 4: Melhor *Random Neurons* (em cada *dataset*) versus *Random Forest* com o mesmo L.

5.3 Síntese dos melhores resultados

- ***Iris*:** melhor *Random Neurons* em $L = 25$, $M = 200$, com acurácia $93{,}89 \pm 3{,}47\,\%$, contra $95{,}56 \pm 2{,}72\,\%$ do *Random Forest*.
- ***Wine*:** melhor *Random Neurons* em $L = 10$, $M = 200$, com acurácia $99{,}07 \pm 1{,}31\,\%$, contra $97{,}45 \pm 2{,}06\,\%$ do *Random Forest*.

6 Discussão crítica

6.1 Comportamento em função de L

Em ambos os *datasets*, a acurácia do *Random Neurons* cresce rapidamente com L nos primeiros valores (de $L = 5$ a $L \approx 25$) e estabiliza em seguida. Esse é o comportamento esperado: o ganho da votação majoritária segue uma curva sub-linear, típica de *ensembles* com componentes pouco correlacionados. A variabilidade entre sementes (barras de erro na Figura 1) também diminui à medida que L cresce, confirmando o efeito estabilizador da agregação.

6.2 Comportamento em função de M

O parâmetro M controla o esforço de otimização de cada classificador base. Para M pequeno (ex.: $M = 10$) o Pocket teve poucas iterações para acumular um bom vetor "de bolso" e o voto majoritário se degrada — perde-se entre 3 e 8 pontos percentuais de acurácia no *Iris* em comparação com $M = 200$. A partir de $M \approx 50$, os ganhos marginais ficam pequenos, e $M = 200$ já se encontra na região de saturação. Em testes exploratórios com $M = 500$ (não reportados nas tabelas), os resultados foram estatisticamente indistinguíveis dos obtidos em $M = 200$, indicando que o Pocket converge rapidamente para um bolso estável nesses *datasets*.

6.3 Iris versus Wine

O contraste entre os dois *datasets* é o resultado mais informativo deste trabalho. O *Wine*, com 13 atributos contínuos e classes essencialmente separáveis por hiperplanos após padronização, é um terreno naturalmente favorável ao *Random Neurons*: o viés indutivo dos Perceptrons se acopla ao problema, e a diversidade gerada pelo *bootstrap* e pela seleção aleatória de atributos

basta para que o voto majoritário corrija erros individuais. O resultado é que o *Random Neurons* supera o *Random Forest* em várias configurações, com a vantagem mais pronunciada em valores pequenos de L (1,6 pontos percentuais em $L = 10$, $M = 200$).

O *Iris*, por outro lado, tem apenas $n = 4$ atributos e uma fronteira ligeiramente não-linear entre as classes *versicolor* e *virginica*. Isso penaliza o *Random Neurons* em duas frentes. Primeiro, com $k \sim \mathcal{U}\{1, \ldots, 4\}$, a probabilidade de sortear $k = 1$ é 25 % — e um único atributo raramente é suficiente para separar três classes. Esses votantes ruins acabam contribuindo com ruído para a agregação. Em segundo lugar, o *Random Forest* captura a não-linearidade entre *versicolor* e *virginica* por meio de partições axiais, enquanto hiperplanos OVR não o fazem. A combinação desses dois efeitos resulta em uma diferença de cerca de 1,7 pontos percentuais a favor do *Random Forest*.

6.4 Pontos fortes do Random Neurons

- **Interpretabilidade geométrica.** Cada votante é um conjunto de c hiperplanos em um subespaço aleatório; o modelo todo é auditável em poucos parâmetros por componente.
- **Custo de predição muito baixo.** Uma inferência equivale, por componente, a um produto interno — interessante em cenários com restrição computacional (*edge computing*).
- **Diversidade "grátis".** A combinação de *bootstrap*, subespaço aleatório e k aleatório garante alta variância entre os votantes, condição necessária para o *bagging* funcionar.

6.5 Limitações

- **Sensibilidade à escala:** exige padronização; sem ela, o Perceptron dá passos desbalanceados entre as dimensões.
- **Limite de expressividade:** não captura fronteiras não-lineares sem engenharia de atributos prévia.
- **Sorteio de k uniforme** em $\{1, \ldots, n\}$ é agressivo em *datasets* com poucos atributos, pois permite componentes com $k = 1$ com probabilidade não desprezível. Uma política alternativa (e.g. $k = \lceil \sqrt{n} \rceil$, análoga ao default do *Random Forest*) poderia melhorar o desempenho base, mas o enunciado pede explicitamente a versão uniforme.